Phase 3 - transactional checkout

Goal: An admitted buyer reserves one ticket and confirms a simulated purchase; expiry releases stock. These operations remain correct under retries and races. Estimated time: 2:45-4:00.

Build

1. Implement transaction-backed reserve, confirm, and expire operations as PostgreSQL functions or explicit SQL transactions. Follow architecture §9. Do not split inventory read and write across API requests.
2. POST /api/events/:id/reserve must check the signed session, event and admission, expiry, one-ticket rule, available stock, and idempotency key/request hash. Atomically move one ticket from available to reserved and create an active reservation. Return the stored response for an identical retry; reject a conflicting reuse of the key.
3. POST /api/reservations/:id/confirm must lock the reservation and event, validate ownership and expiry, atomically move one ticket from reserved to sold, and create one order. A repeated confirmation returns the same order.
4. Implement due-hold expiry with reservation row locks and state guards. Atomically move stock from reserved back to available. Call expiry before relevant admissions/reservations and expose an organizer trigger for the demo. Confirm and expire must contend on the same reservation row.
5. Build buyer reservation timer and simulated checkout. Use server expiry timestamps; refresh from /me before interpreting result. Show confirmed order ID or clear expired/sold-out state.
6. Add focused concurrency tests for last-ticket reserve, duplicate confirm, confirm versus expiry, and reconnect during an active hold.

Acceptance checks

- Two simultaneous reserve calls for the final ticket yield one active reservation, zero oversells, and valid inventory totals.
- Two simultaneous confirmations yield one order and one transfer from reserved to sold.
- Confirm versus expire ends with either one order or one released ticket, never both.
- Identical retries return the same result; conflicting idempotency-key reuse is rejected.
- Refresh/reconnect shows the same reservation or order. A buyer cannot reserve for another account.

Handoff to Phase 4

Phase 4 needs reliable counters and decision records. Record transaction function names, idempotency behavior, expiry trigger requirements, test command/results, and any missing scheduler limitation.

Handoff record

Fill in after implementation: files/commits, commands, results, and open issues.